

Programowanie systemowe

Sprawozdanie

Aron Krajda - 283874

Data wykonania sprawozdania: 12.01.2026

Cele:

Celem laboratorium była dalsza modyfikacja plików, tym razem należało użyć pamięci współdzielonej i semaforów oraz zapoznać się z ich działaniem.

Plik main.c:

W pliku main.c nie wprowadzono żadnych zmian. Pozostaje on funkcjonalny w poprzedniej formie:

```
#include <stdio.h>
#include <string.h>
#include "server.h"
#include "worker.h"

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Użycie: %s [server|worker]\n", argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "server") == 0) {
        server();
    } else if (strcmp(argv[1], "worker") == 0) {
        worker();
    } else {
        printf("Użyj: server lub worker\n");
        return 1;
    }

    return 0;
}
```

Plik server.c:

Plik server.c zmodyfikowano tak, aby używał pamięci współdzielonej i semaforów:
Do pliku server.c dodano definicje struktury danych pamięci współdzielonej (typedef struct...) w tym semafony kontrolujące dostęp servera/workera.

Dodano również usuwanie starej pamięci współdzielonej aby zapewnić pustą pamięć współdzieloną przy uruchamianiu programu (shm_unlink("/shm");).

Kolejno zaimplementowano tworzenie pamięci współdzielonej o nazwie "/shm", ustawiono jej rozmiar, zmapowano pamięć do przestrzeni adresowej procesu i zainicjowano semafony (fragment kodu od int fd =... do sem_init...).

Następnie w kodzie znajduje się pętla while działająca podobnie jak w poprzednim laboratorium, jednak korzysta teraz z pamięci współdzielonej i semaforów:

Na końcu pliku znajduje się fragment niszczący powstałe semafony, odmapowujący pamięć z procesu i usuwający pamięć współdzieloną:

```
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <semaphore.h>

typedef struct {
    char data[256];
    sem_t sem_server;
    sem_t sem_worker;
} mem_t;

void server() {
    char input[256];

    shm_unlink("/shm");
    int fd = shm_open("/shm", O_CREAT | O_RDWR, 0666);
    ftruncate(fd, sizeof(mem_t));
    mem_t *m = mmap(0, sizeof(mem_t), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    sem_init(&m->sem_server, 1, 1);
    sem_init(&m->sem_worker, 1, 0);

    while (1) {
        scanf("%s", input);

        sem_wait(&m->sem_server);
        strcpy(m->data, input);
        sem_post(&m->sem_worker);

        if (strcmp(input, "exit") == 0) break;

        sem_wait(&m->sem_server);
        printf("%s\n", m->data);
        sem_post(&m->sem_worker);
    }

    sem_destroy(&m->sem_server);
    sem_destroy(&m->sem_worker);
    munmap(m, sizeof(mem_t));
    close(fd);
    shm_unlink("/shm");
}
```

Plik worker.c:

Plik worker.c również zmodyfikowano tak, aby używał pamięci współdzielonej i semaforów:

Zdefiniowano identyczną strukturę danych pamięci współdzielonej co w pliku server.c.

Dodano otwieranie pamięci współdzielonej przez worker (int fd = ...).

Dodano mapowanie tej samej pamięci co server (mem_t *m = ...).

Następnie w kodzie znajduje się zmodyfikowana względem poprzednich laboratoriów pętla while, która podobnie jak w przypadku pliku server.c korzysta teraz z pamięci współdzielonej i semaforów.

Na koniec pamięć jest odmapowywana:

```
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <semaphore.h>

typedef struct {
    char data[256];
    sem_t sem_server;
    sem_t sem_worker;
} mem_t;

void worker() {
    int fd = shm_open("/shm", O_RDWR, 0666);
    mem_t *m = mmap(0, sizeof(mem_t), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    while (1) {
        sem_wait(&m->sem_worker);

        if (strcmp(m->data, "exit") == 0) {
            sem_post(&m->sem_server);
            break;
        }

        m->data[0] = 'X';
        sem_post(&m->sem_server);
    }

    munmap(m, sizeof(mem_t));
    close(fd);
}
```

Uruchomienie i wnioski:

Po uruchomieniu programu działa on poprawnie wykonując nadane mu wcześniej zadanie, jednak tym razem zamiast komunikacji mkfifo korzysta z pamięci współdzielonej oraz wykorzystuje semaforey.

Przy tworzeniu programu zauważyć można było, że semaforey są konieczne do poprawnego działania tak skonstruowanego programu. Zapobiegają one konfliktom przy dostępie do pamięci współdzielonej, a bez tego program nie byłby w stanie działać poprawnie, na co wskazało uruchomienie programu bez użycia semafor w czasie laboratoriów jak polecił prowadzący.

```
vboxuser@ubuntu22:~/Desktop/LAB9$ make run_server
gcc -Wall -c main.c
gcc -Wall -c server.c
gcc -Wall -o program main.o server.o worker.o -lrt -lpthread
./program server
dad
Xad
exit
vboxuser@ubuntu22:~/Desktop/LAB9$ 
vboxuser@ubuntu22:~/Desktop/LAB9$ make run_worker
./program worker
vboxuser@ubuntu22:~/Desktop/LAB9$
```